UNIVERSIDAD DE ALCALÁ

Grado en Ingeniería en Sistemas de Información (GISI)

Ingeniería del Software - Curso 2025/2026

PECL 2: Laboratorio

Pruebas de Software y Análisis de Métricas

Autores

- Gonzalo Rodríguez Castro
- David Salgado Soltero

Fecha: 06/05/2026

Índice

1. Ejercicio 1: Diseño de las pruebas para el método binarySearch
    - 1.1 Complejidad Ciclomática
    - 1.2 Tabla de casos de prueba
2. Ejercicio 2: Diseño de los casos de prueba para el método evaluarCliente
    - 2.1 Tabla de clases de equivalencia y valores límite
3. Ejercicio 3: Análisis de métricas de JChecs
    - 3.1 Análisis con JavaNCSS y CKJM
    - 3.2 Valoración de mantenimiento

Ejercicio 2

Complejidad Ciclómatica

```
public class BinarySearch {
    //pueblic static void main(Strings [] args){}
    public static int binarySearch(int[] A, int x) {
// search space is `A[left...right]`
        int left = 0, right = A.length - 1;
// loop till the search space is exhausted
        while (left <= right) {
// find the mid-value in the search space and
// compares it with the target
            int mid = (left + right) / 2;
// key is found
            if (x == A[mid]) {
                return mid;
            } // discard all elements in the right search space,
            // including the middle element
            else if (x < A[mid]) {
                right = mid - 1;
            } // discard all elements in the left search space,
            // including the middle element
            else {
                left = mid + 1;
            }
        }
// `x` doesn't exist in the array
        return -1;
    }
}
```

Para el método binarySearch, la complejidad la calculamos contando las decisiones:

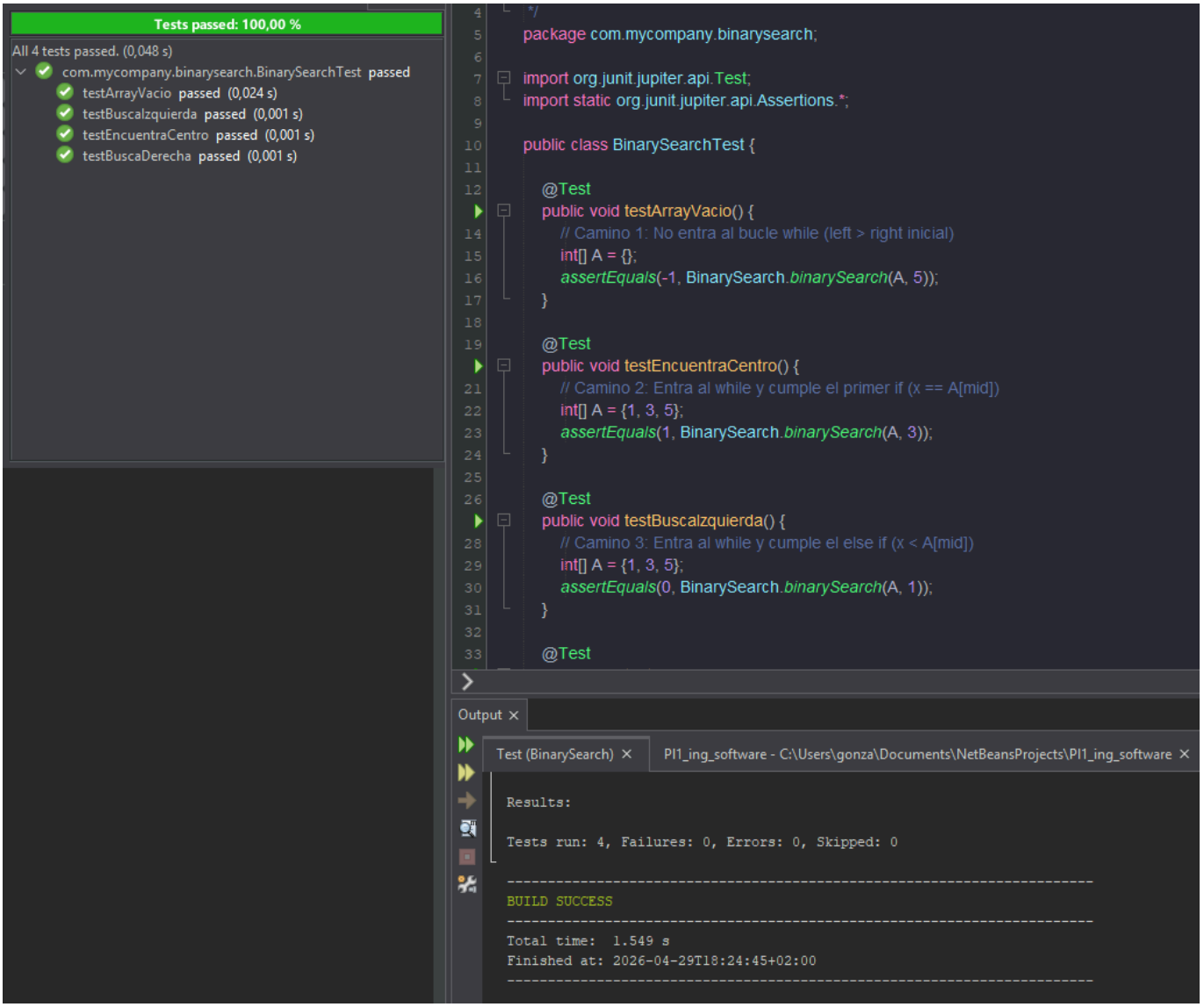
Decisiones: 1 while, 1 if, 1 Else if = 1 + 1 + 1 = 3

Fórmula: nº decisiones + 1 = 4.

Cómo el resultado de la fórmula de la complejidad ciclomática es 4 sabesmos que al menos necesitamos al menos 4 cassos de prueba para poder cubrir todos los caminos independientes.

Tabla de casos de prueba

ID	Valor X	Array A	Resultado esperado
CP1	5	{}	-1
CP2	3	{1,3,5}	1
CP3	1	{1,3,5}	0
CP4	5	{1,3,5}	2



Ejercicio 3

Para este ejercicio utilizamos la técnica de Clases de equivalencia y valores límite sobre el método dentro de cajero automático.

Tabla de clases de equivalencia

```java
public boolean evaluarCliente(double cantidad, int plazo, int edad, GestorHipoteca gestor) {
    assert (_cuenta != null);
    if (!gestor.comprobarCantidad(cantidad)) {
        return false;
    }
    if (!gestor.comprobarPlazo(plazo)) {
        return false;
    }
    if (!gestor.comprobarEdad(edad)) {
        return false;
    }
    gestor.abrirHipoteca(cantidad, plazo);
    return true;
}
```

Parámetro	Clase válida	Clases inválidas
Cantidad	0<= x <= 200.000	x > 200.000
Plazo	0<= x <= 240	x > 240
Edad	x <= 50	x > 50

Sabemos que los parametros que tenemos que testear son 3 y tienen una opción válida y una inválida, entoces para calcular la cantidad de pruebas que tenemos que realizar podríamos calcularla de tal forma 2^3 = 8 TEST.

com.mycompany:CajeroAutomatico:jar:1.0-SNAPSHOT (Unit)

Tests passed: 100,00 %

All 8 tests passed. (0,078 s)

- com.mycompany.cajeroautomatico.CajeroAutomaticoTest
  - testEvaluarCliente_EdadInvalida passed (0,039 s)
  - testEvaluarCliente_CantidadInvalida passed (0,003 s)
  - testRealizarRetirada_SaldoSuficiente passed (0,002 s)
  - testRealizarRetirada_SaldoInsuficiente passed (0,001 s)
  - testEvaluarCliente_PlazoInvalido passed (0,001 s)
  - testRealizarRetirada_CuentaBloqueadaConSaldo passed
  - testEvaluarCliente_HipotecaValida passed (0,0 s)
  - testRealizarRetirada_CuentaBloqueadaSinSaldo passed

Source History

```
28          EasyMock.replay(mockPasarela);
29
30          boolean resultado = cajero.realizarRetirada(100.0, mockPasarela);
31
32          assertTrue(resultado);
33          EasyMock.verify(mockPasarela);
34      }
35
36      @Test
        public void testRealizarRetirada_SaldoInsuficiente() {
38          cajero.iniciarSesion("ES1234567890");
39
40          EasyMock.expect(mockPasarela.estaBloqueada()).andReturn(false);
41          EasyMock.expect(mockPasarela.tieneSaldo(100.0)).andReturn(false);
42          EasyMock.replay(mockPasarela);
43
44          boolean resultado = cajero.realizarRetirada(100.0, mockPasarela);
45
46          assertFalse(resultado);
47          EasyMock.verify(mockPasarela);
48      }
49
50      @Test
        public void testRealizarRetirada_CuentaBloqueadaConSaldo() {
52          cajero.iniciarSesion("ES1234567890");
53
54          EasyMock.expect(mockPasarela.estaBloqueada()).andReturn(true);
55          EasyMock.replay(mockPasarela);
56
57          boolean resultado = cajero.realizarRetirada(100.0, mockPasarela);
```

com.mycompany.cajeroautomatico.CajeroAutomaticoTest > testRealizarRetirada_SaldoSuficiente

Output

PI1_ing_software - C:\Users\gonza\Documents\NetBeansProjects\PI1_ing_software | Test (CajeroAutomatico)

```
Results:

Tests run: 8, Failures: 0, Errors: 0, Skipped: 0

------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.491 s
Finished at: 2026-04-29T19:17:02+02:00
------------------------------------------------------------------------
```